

Vinicius Raphael Ulian

RA: 824147726

Aula 05 - Modelo físico e integridade no MySQL

Exercício 01 - Escolha tipos para um sistema acadêmico

Atributo	Tipo Físico (MySQL)	Justificativa
id	INT AUTO_INCREMENT PRIMARY KEY	Identificador substituto interno.
matrícula	CHAR(10) UNIQUE NOT NULL	Código com tamanho fixo que não sofre operações aritméticas (preserva zeros à esquerda). Identificador alternativo.
nome	VARCHAR(150) NOT NULL	Texto curto de comprimento variável; obrigatório.
e-mail	VARCHAR(254) UNIQUE	Texto curto variável. Limite padrão RFC. UNIQUE protege o identificador alternativo.
nascimento	DATE NOT NULL	Representa uma data civil sem horário, fuso horário não importa neste contexto.
coeficiente	DECIMAL(4,2)	Preserva precisão exata para notas/médias (ex: 10.00).
bolsista	BOOLEAN NOT NULL DEFAULT FALSE	Estado lógico (Verdadeiro/Falso). No MySQL é convertido para TINYINT(1).

observações	TEXT	Guarda conteúdo extenso e livre onde VARCHAR deixa de ser natural.
--------------------	------	--

Identificação de estado dos dados:

- **Dado Obrigatório:** nome e matrícula (NOT NULL).
- **Dado Ausente:** e-mail ou observações (podem ser NULL se o aluno não informou).
- **Dado Inaplicável:** coeficiente para um aluno recém-matriculado que ainda não possui notas (pode ser NULL).

Exercício 2 - Declare a tabela RESERVA com constraints coerentes

```
CREATE TABLE reserva (
  id_reserva INT NOT NULL AUTO_INCREMENT,
  id_sala INT NOT NULL,
  id_responsavel INT NOT NULL,
  inicio DATETIME NOT NULL,
  fim DATETIME NOT NULL,
  situacao VARCHAR(20) NOT NULL DEFAULT 'CONFIRMADA',

  CONSTRAINT pk_reserva PRIMARY KEY (id_reserva),
  CONSTRAINT ck_reserva_tempo CHECK (fim > inicio),
  CONSTRAINT ck_reserva_situacao CHECK (situacao IN ('CONFIRMADA',
'CANCELADA', 'CONCLUIDA')),
  CONSTRAINT fk_reserva_sala FOREIGN KEY (id_sala)
  REFERENCES sala(id_sala) ON DELETE RESTRICT,
  CONSTRAINT fk_reserva_resp FOREIGN KEY (id_responsavel)
  REFERENCES responsavel(id_responsavel) ON DELETE RESTRICT
) ENGINE = InnoDB;
```

Porque UNIQUE(id_sala, inicio) não detecta toda sobreposição temporal?

- A restrição UNIQUE avalia apenas se há igualdade exata de valores. Ela impediria duas reservas começando no mesmo exato segundo, mas permitiria que uma reserva começasse às 10:00 e terminasse às 12:00, e outra na mesma sala começasse às 10:30, gerando um conflito no mundo real não detectado pela constraint física.

Exercício 03 - Escolha ON DELETE e ON UPDATE para três vínculos

- **DEPARTAMENTO → EMPREGADO:** O empregado pode permanecer sem departamento? Se sim (vínculo opcional): **ON DELETE SET NULL**. Mantém o filho, mas remove o vínculo.

- **PROJETO → TAREFA:** A tarefa possui existência independente? Não. Portanto, se o projeto for excluído: **ON DELETE CASCADE**. A tarefa é parte inseparável do projeto.
- **AUTOR → LIVRO_AUTOR:** O contrato histórico pode ser apagado? Não. A autoria é um fato histórico: **ON DELETE RESTRICT**. Impede a exclusão do autor se houver vínculos.
- **ON UPDATE (Para todos):** Como assumimos o uso de chaves primárias substitutas (INT AUTO_INCREMENT), elas devem ser estáveis. Portanto, a regra recomendada é **ON UPDATE RESTRICT**.

Exercício 04 - Projete violações que demonstrem a eficácia das constraints

-- 1. Violação de PRIMARY KEY (Identidade repetida)

```
INSERT INTO paciente (id_paciente, nome) VALUES (1, 'Ana');
```

```
INSERT INTO paciente (id_paciente, nome) VALUES (1, 'Bruno');
```

-- Erro: Duplicate entry '1' for key 'PRIMARY'

-- 2. Violação de UNIQUE e CHECK

```
INSERT INTO paciente (cpf, email) VALUES ('12345678901', 'ana@email.com');
```

-- Tentativa de reusar email:

```
INSERT INTO paciente (cpf, email) VALUES ('98765432100', 'ana@email.com');
```

-- Erro de UNIQUE

-- Tentativa de ferir CHECK de quantidade (ex: CHECK quantidade > 0):

```
INSERT INTO item_pedido (id_pedido, quantidade) VALUES (1, -5);
```

-- Erro: Check constraint is violated

-- 3. Inserir FK Órfã

```
INSERT INTO pedido (id_cliente, total) VALUES (9999, 150.00);
```

-- Erro: Cannot add or update a child row: a foreign key constraint fails

-- 4. Exclusão bloqueada por RESTRICT

```
DELETE FROM cliente WHERE id_cliente = 1;
```

-- Erro se o cliente tiver pedidos, bloqueado por RESTRICT

-- 5. Exclusão propagada por CASCADE

```
DELETE FROM pedido WHERE id_pedido = 10;
```

-- Sucesso. Estado final: O pedido 10 é apagado de PEDIDO e todas as linhas

-- correspondentes na tabela ITEM_PEDIDO também são excluídas.

Exercício 05 - Encontre redundâncias e anomalias em uma relação não normalizada

Tabela Não Normalizada: VENDA(id_venda, data, id_cliente, nome_cliente, id_produto, nome_produto, quantidade).

Fatos misturados: Informações da venda, informações do cliente e informações do produto.

- **Anomalia de Atualização:** Se o cliente alterar o nome, o banco precisará atualizar o nome_cliente em todas as linhas de suas vendas passadas. Uma falha gera versões conflitantes.
- **Anomalia de Inserção:** Não é possível cadastrar um produto novo (id_produto, nome_produto) antes que ocorra uma venda, pois faltariam dados para id_venda e id_cliente (que não podem ser nulos).
- **Anomalia de Exclusão:** Se a última venda de um produto específico for apagada, todas as informações sobre aquele produto (nome) desaparecerão do banco de dados.

Decomposição e Correção: Separar entidades independentes e conectá-las por chaves estrangeiras.

CLIENTE(id_cliente PK, nome_cliente)

PRODUTO(id_produto PK, nome_produto, preco_atual)

VENDA(id_venda PK, data, id_cliente FK)

ITEM_VENDA(id_venda PK/FK, id_produto PK/FK, quantidade, preco_praticado)

Atributo Histórico: O preco_praticado deve permanecer em ITEM_VENDA para não ser afetado pelas variações do preco_atual em PRODUTO.

Desafio Integrador: Implemente o modelo físico de uma clínica

```
CREATE SCHEMA IF NOT EXISTS clinica
```

```
CHARACTER SET utf8mb4
```

```
COLLATE utf8mb4_0900_ai_ci;
```

```
USE clinica;
```

```
CREATE TABLE paciente (
```

```
    id_paciente INT NOT NULL AUTO_INCREMENT,
```

```
    cpf CHAR(11) NOT NULL,
```

```
    nome VARCHAR(120) NOT NULL,
```

```
    data_nascimento DATE NOT NULL,
```

```
    CONSTRAINT pk_paciente PRIMARY KEY (id_paciente),
```

```
    CONSTRAINT uq_paciente_cpf UNIQUE (cpf)
) ENGINE = InnoDB;
```

```
CREATE TABLE medico (
    id_medico INT NOT NULL AUTO_INCREMENT,
    crm VARCHAR(20) NOT NULL,
    nome VARCHAR(120) NOT NULL,
    especialidade VARCHAR(80) NOT NULL,
    CONSTRAINT pk_medico PRIMARY KEY (id_medico),
    CONSTRAINT uq_medico_crm UNIQUE (crm)
) ENGINE = InnoDB;
```

```
CREATE TABLE procedimento (
    id_procedimento INT NOT NULL AUTO_INCREMENT,
    descricao VARCHAR(150) NOT NULL,
    valor_base DECIMAL(10,2) NOT NULL,
    CONSTRAINT pk_procedimento PRIMARY KEY (id_procedimento),
    CONSTRAINT ck_proced_valor CHECK (valor_base >= 0)
) ENGINE = InnoDB;
```

```
CREATE TABLE consulta (
    id_consulta INT NOT NULL AUTO_INCREMENT,
    id_paciente INT NOT NULL,
    id_medico INT NOT NULL,
    data_hora DATETIME NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'AGENDADA',
    CONSTRAINT pk_consulta PRIMARY KEY (id_consulta),
    CONSTRAINT ck_consulta_status CHECK (status IN ('AGENDADA', 'REALIZADA',
'CANCELADA')),
    CONSTRAINT fk_consulta_paciente FOREIGN KEY (id_paciente)
        REFERENCES paciente(id_paciente) ON DELETE RESTRICT ON UPDATE
        RESTRICT,
    CONSTRAINT fk_consulta_medico FOREIGN KEY (id_medico)
        REFERENCES medico(id_medico) ON DELETE RESTRICT ON UPDATE RESTRICT
) ENGINE = InnoDB;
```

```
CREATE TABLE consulta_procedimento (
```

```

id_consulta INT NOT NULL,
id_procedimento INT NOT NULL,
valor_cobrado DECIMAL(10,2) NOT NULL,
CONSTRAINT pk_cons_proc PRIMARY KEY (id_consulta, id_procedimento),
CONSTRAINT ck_cons_proc_valor CHECK (valor_cobrado >= 0),
CONSTRAINT fk_cp_consulta FOREIGN KEY (id_consulta)
    REFERENCES consulta(id_consulta) ON DELETE CASCADE ON UPDATE
RESTRICT,
CONSTRAINT fk_cp_procedimento FOREIGN KEY (id_procedimento)
    REFERENCES procedimento(id_procedimento) ON DELETE RESTRICT ON UPDATE
RESTRICT
) ENGINE = InnoDB;

```

Justificativas de Integridade (Ciclo de Vida):

- **ON DELETE RESTRICT** nas FKs de CONSULTA impede exclusão acidental de pacientes ou médicos que possuam consultas registradas (histórico médico legal).
- **ON DELETE CASCADE** na FK id_consulta dentro de CONSULTA_PROCEDIMENTO garante que, se uma consulta for cancelada e removida (exclusão consciente), os procedimentos atrelados a ela, que não existem independentemente, também sejam excluídos.